Q1) You are monitoring login attempts on a system. Each login attempt contains:

• A username

• A login status (success or fail) If a user fails to log in 3 times, this user should be flagged as suspicious.

logins = [("ahmed", "success"), ("root", "fail"), ("root", "fail"), ("root", "fail"), ("sara", "success"), ("root", "fail"), ("ahmed", "fail")]

Tasks:

• Track failed login attempts per user

• Print an alert when a user reaches 3 failed attempts

In [ ]:
```
def login_attempts(logins, name):
    """Monitors login attempts for a specific user and alerts on 3 failed attempts"""
    flag = 0
    for user, status in logins:
        if status == "fail":
            flag += 1
    print(f"failed attempts: {flag}")

    if flag >= 3:
        print("Alert! 3 fail attempts suspicious!")
    elif flag == 0:
        print(f"No fail attempts for {name}")
    elif flag <3:
        print(f"Warning! {flag} fail attempts! for {name}")


logins = [
    ("ahmed", "success"),
    ("root", "fail"),
    ("root", "fail"),
    ("root", "fail"),
    ("sara", "success"),
    ("root", "fail"),
    ("ahmed", "fail")
]
login_attempts(logins, "sara")
login_attempts(logins, "ahmed")
login_attempts(logins, "root")
```

```
failed attempts: 0
No fail attempts for sara
failed attempts: 1
Warning! 1 fail attempts! for ahmed
failed attempts: 4
Alert! 3 fail attempts suspicious!
```

Q2: Log Analyzer You are analyzing system log messages. Each log line starts with a log level such as:

• INFO

• WARNING

• ERROR

logs = ["INFO system boot", "ERROR disk full", "WARNING memory high", "ERROR permission denied", "INFO service started", "WARNING cpu high"] Tasks:

- Count how many messages exist for each log level

- Print only ERROR and WARNING messages

- Print a summary of all log levels and their counts

In [16]:
```
def log_analyzer(logs):
    """Count how many messages exist for each log level """
    count = {"INFO" : 0 , "ERROR" : 0 , "WARNING" : 0}

    for i in range(len(logs)):
        log_level = (logs[i].split())[0]
        if log_level in count:
            count[log_level] += 1
    return count

def log_notification(logs):
    """print only ERROR and WARNING messages"""
    for i in range(len(logs)):
        log_level = (logs[i].split())[0]
        if log_level == "ERROR" or log_level == "WARNING":
            print(f"{logs[i]}")


def log_summary(logs):
    """Print a summary of all log levels and their counts"""
    count = log_analyzer(logs)
    for key, value in count.items():
        print(f"{key}: {value}")


logs = [
    "INFO system boot",
    "ERROR disk full",
    "WARNING memory high",
    "ERROR permission denied",
    "INFO service started",
    "WARNING cpu high"
]

print(f"{log_analyzer(logs)}\n")
log_notification(logs)
print("\n")
log_summary(logs)
```

```
{'INFO': 2, 'ERROR': 2, 'WARNING': 2}

ERROR disk full
WARNING memory high
ERROR permission denied
WARNING cpu high


INFO: 2
ERROR: 2
WARNING: 2
```

Q3: Disk Cleanup Decision Engine You are monitoring disk usage on a server.

disks = { "/": 85, "/home": 70, "/var": 92, "/tmp": 60 }

Rules:

- Usage > 90 → CRITICAL cleanup required

• Usage 80–90 → WARNING

• Usage < 80 → OK

Tasks:

• Print the status of each disk

• Count how many disks need cleanup

In [ ]:
```
def disk_usage(disks):
    """Check disk usage"""
    count = 0
    for disk, usage in disks.items():
        if usage >= 90:
            count += 1
            print(f"CRITICAL cleanup required!  disk ({disk}) usage at {usage}%")
        elif usage > 80 and usage < 90:
            print(f"Warning!  disk ({disk}) usage at {usage}%")
        elif usage <= 80:
            print(f"OK!  disk ({disk}) usage at {usage}%")
    print(f"\nTotal disks need cleanup: {count}")

disks = {
    "/": 85,
    "/home": 70,
    "/var": 92,
    "/tmp": 60
}

disk_usage(disks)
```

```
Warning!  disk (/) usage at 85%
OK!  disk (/home) usage at 70%
CRITICAL cleanup required!  disk (/var) usage at 92%
OK!  disk (/tmp) usage at 60%

Total disks need cleanup: 1
```

Q4: Command Execution Simulator You are simulating command execution results.

Each command has:

• Command text

• Exit code (0 means success, non-zero means failure)

commands = [("ls", 0), ("mkdir test", 0), ("rm important.txt", 1), ("cp file1 file2", 0)]

Tasks:

• Print whether each command succeeded or failed

• Count the number of failed commands

In [22]:
```
def command_monitor(commands):
    """Monitor each command whether each command succeeded or failed
     & Count the number of failed commands """
    failed_count = 0
    for command, code in commands:
        if code != 0:
            print(f"Command '{command}' failed with status {code}")
            failed_count += 1
```

```
        else:
            print(f"Command '{command}' executed successfully.")

    print(f"\nTotal failed commands: {failed_count}")



commands = [
    ("ls", 0),
    ("mkdir test", 0),
    ("rm important.txt", 1),
    ("cp file1 file2", 0)
]
command_monitor(commands)
```

```
Command 'ls' executed successfully.
Command 'mkdir test' executed successfully.
Command 'rm important.txt' failed with status 1
Command 'cp file1 file2' executed successfully.

Total failed commands: 1
```

Q5: Resource Abuse Detector You are monitoring CPU usage for running processes.

Note: Multiple entries may exist for the same process.

processes = [("nginx", 30), ("python", 95), ("mysql", 85), ("redis", 20), ("python", 90)]

Tasks:

- Calculate total CPU usage per process
- If total CPU usage > 100%, print an alert
- Print only processes that exceed the limit

In [35]:

```
def cpu_monitor(processes):
    """Calculate total CPU usage per process and alert if CPU usage > 100% and print this proc
    process_cpu = {}
    for process, cpu in processes:
        if process in process_cpu:
            process_cpu[process] += cpu
        else:
            process_cpu[process] = cpu
    return process_cpu

def print_exceed_limit(processes):
    """Print processes that exceed CPU limit of 100%"""
    process_cpu = cpu_monitor(processes)
    for process, cpu in process_cpu.items():
        if cpu >= 100:
            print(f"ALERT: Process '{process}' CPU usage is {cpu}%")


processes = [
    ("nginx", 30),
    ("python", 95),
    ("mysql", 85),
    ("redis", 20),
    ("python", 90)
]
print(cpu_monitor(processes))
print_exceed_limit(processes)
```

```
{'nginx': 30, 'python': 185, 'mysql': 85, 'redis': 20}
ALERT: Process 'python' CPU usage is 185%
```